

Module - II

①a) Context Free Grammar

• It is a 4-tuple $G = (V, \Sigma, P, S)$ where:

- V is finite set of non terminals
- Σ is finite set of terminals
- P is finite set of production rules of form $A \rightarrow a$.

where, $A \in V$ & $a \in (V \cup \Sigma)^*$.

- $S \in V$ is start symbol.

• The language generated by CFG is:

$L = \{a^n b^n \mid n \geq 0\}$, i.e., strings of n a's followed by n b's.

Ex: $P : \{S \rightarrow asb \mid S \rightarrow \epsilon\}$.

Step-1: Consider Grammar $G = \{V, \Sigma, P, S\}$:

$V = \{S\}$

$\Sigma = \{a, b\}$

$P = \{S \rightarrow asb \mid S \rightarrow \epsilon\}$.

S = start symbol.

Step-2: Derive the string

$S \Rightarrow asb \quad (S \rightarrow asb)$

$\Rightarrow aasbb \quad (S \rightarrow asb)$

$\Rightarrow aabb. \quad (S \rightarrow \epsilon).$

• Ex: $\epsilon, ab, aabb, aaabbb, aaaa bbbb, \dots$

①b) The strings over $\{0, 1\}$ which do not contain equal no. of 0's & 1's.

- String having more 0's than 1's.
- String having more 1's than 0's.

- If we generate all strings from both types and take their union, we get required strings

Step-1: Grammar

$$S \rightarrow P \mid N$$

$$E \rightarrow OEIE \mid IE OE \mid \epsilon$$

$$P \rightarrow EOE \mid EOP$$

$$N \rightarrow EIE \mid EIN$$

S is the start symbol

E is generates string with equal no. of 0's & 1's.

P is generates strings with more 0's than 1's.

N is generates strings with more 1's than 0's.

Non-terminal E : $E \rightarrow OEIE \mid IE OE \mid \epsilon$

- It is balanced part because all strings have equal no. of 0's & 1's.

Eg: $\epsilon, 01, 10, 0011, 0101, 1001, \dots$ ($0=1$).

Non-terminal P : $P \rightarrow EOE \mid EOP$

- Here, an extra 0 is introduced with balanced strings around it.

$0 > 1$ in all strings generated by P .

Eg: $0, 00, 001, 010, 00011, 10010, \dots$ ($0 > 1$).

Non-terminal N : $N \rightarrow EIE \mid EIN$

- Here, an extra 1 is introduced with balanced strings around it.

$1 > 0$ in all strings generated by N .

Eg: $1, 11, 101, 011, 11100, 001111, \dots$ ($1 > 0$).

Every string over $\{0, 1\}$ in which no. of 0's & 1's is not equal must belong to exactly one of above two cases.

$$\therefore \boxed{S \rightarrow P \mid N}$$

② a) CFG for all palindromes over a, b

- palindrome is a string that reads same from left to right & right to left.

Eg: $\epsilon, a, b, aa, bb, aba, abba, \dots$

Grammar: $S \rightarrow asa / bsb / a / b / \epsilon$

- $S \rightarrow asa$: adds a to both ends, preserving the palindrome property.
- $S \rightarrow bsb$: adds b to both ends, preserving the palindrome property.
- $S \rightarrow a$: generates a single character palindrome.
- $S \rightarrow b$: generates a single character palindrome.
- $S \rightarrow \epsilon$: generates the empty string.

Derivation:

$$S \Rightarrow asa$$

$$S \Rightarrow absba$$

$$S \Rightarrow abba$$

$\therefore$ Required CFG is:

$$S \rightarrow asa / bsb / a / b / \epsilon$$

② b) Derivation:

- It is the process of generating a string from the start symbol of CFG by repeatedly applying the production rules.

Eg: $S \rightarrow asa / bsb / a / b / \epsilon$

$$S \Rightarrow asa$$

$$\Rightarrow absba$$

$$\Rightarrow abba$$

(arrow represents the application of production rule).

(i) Leftmost Derivation (lm)

- At every step, we replace the leftmost non-terminal in the current string.

Eg: $S \rightarrow asb / ab$

$$S \Rightarrow asb$$

$$S \Rightarrow aabb$$

$\therefore$ S is always expanded from leftmost position

(2) Rightmost Derivation (rm)

- At every step, we replace the rightmost non-terminal in current string.

Eg: $S \rightarrow AB$, $A \rightarrow aA/a$, $B \rightarrow bB/b$.

$$S \Rightarrow AB$$

(Rightmost non-terminal is B)

$$S \Rightarrow AbB$$

(Rightmost non-terminal is B)

$$S \Rightarrow Abb$$

(Rightmost non-terminal is A)

$$S \Rightarrow aAbb$$

$$\boxed{S \Rightarrow aabb}$$

Derivation Tree / Parse Tree

- It is a tree representation of how a string is generated from CFG.

(1) The root is the start symbol.

(2) Internal nodes are non-terminals.

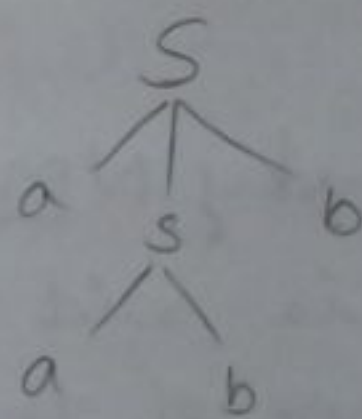
(3) Leaf nodes are terminals or ϵ .

(4) Reading the leaf nodes from left to right gives generated string.

Eg: $S \rightarrow asb/ab$.

$$S \Rightarrow asb$$

$$S \Rightarrow aabb$$



③ a) Given CFG:

$$S \rightarrow 0B/1A$$

$$A \rightarrow 0/0S/1AA$$

$$B \rightarrow 1/1S/0BB$$

Given string: $w = 00110101$.

i) Leftmost

$$S \Rightarrow OB$$

$$S \Rightarrow OOB B \quad (B \rightarrow OBB)$$

$$S \Rightarrow OOB B \quad (B \rightarrow 1)$$

$$S \Rightarrow OOB 1 S \quad (B \rightarrow 1S)$$

$$S \Rightarrow OOB 1 OB \quad (S \rightarrow OB)$$

$$S \Rightarrow OOB 1 O 1 S \quad (B \rightarrow 1S)$$

$$S \Rightarrow OOB 1 O 1 OB \quad (S \rightarrow OB)$$

$$S \Rightarrow OOB 1 O 1 O 1 \quad (B \rightarrow 1)$$

ii) Rightmost

$$S \Rightarrow OB$$

$$S \Rightarrow OOB B \quad (B \rightarrow OBB)$$

$$S \Rightarrow OOB 1 S \quad (B \rightarrow 1S)$$

$$S \Rightarrow OOB 1 OB \quad (S \rightarrow OB)$$

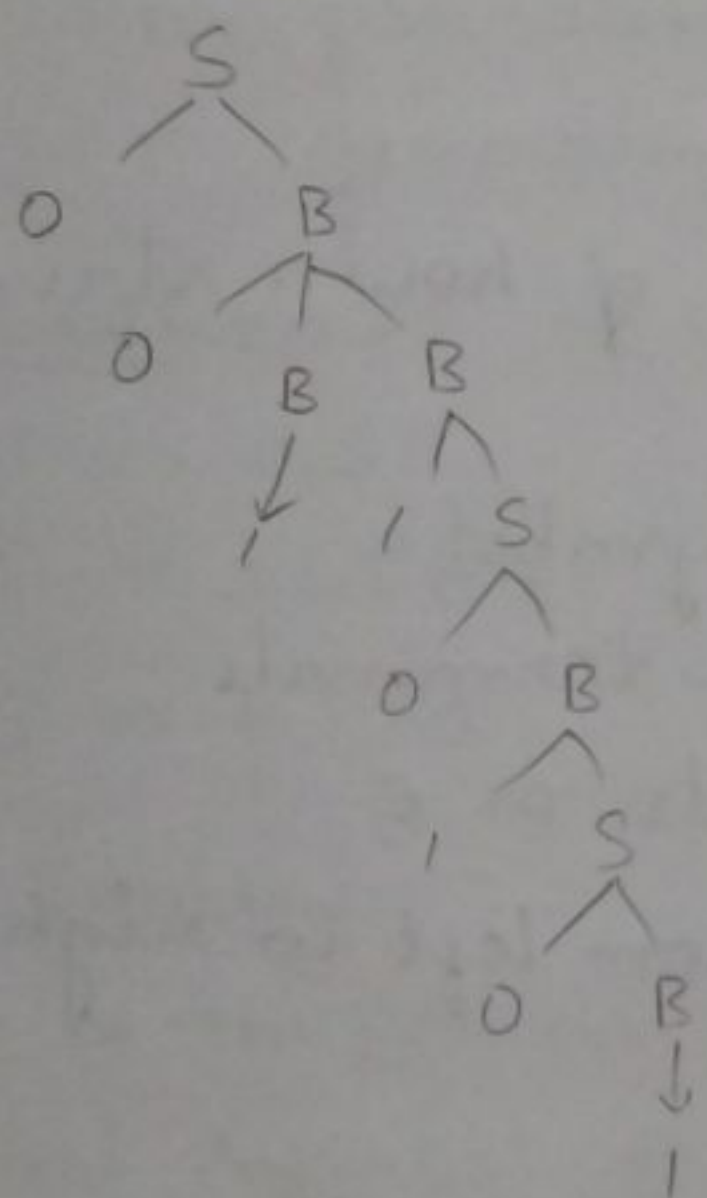
$$S \Rightarrow OOB 1 O 1 S \quad (B \rightarrow 1S)$$

$$S \Rightarrow OOB 1 O 1 OB \quad (S \rightarrow OB)$$

$$S \Rightarrow OOB 1 O 1 O 1 \quad (B \rightarrow 1)$$

$$S \Rightarrow OOB 1 O 1 O 1 \quad (B \rightarrow 1)$$

iii) Derivation Tree



Reading the leaf nodes from left to right:

00110101

③ b) Chomsky Normal Form (CNF).

- A CFG is said to be in CNF, if every production is of one of the following forms:

$$A \rightarrow BC \quad (\text{or}) \quad A \rightarrow a.$$

where, A, B, C are non terminals
a is terminals.

Conversion of CFG to CNF

Step-1: Eliminate ϵ productions

Step-2: Eliminate unit productions

Step-3: Replace terminals in mixed productions

Step-4: Convert long productions into binary productions.

eg: $S \rightarrow axbX$
 $X \rightarrow aY | bY | \epsilon$
 $Y \rightarrow X | c$

Step-1: Eliminate ϵ -productions

- From $X \rightarrow \epsilon$, X is nullable.
- In $S \rightarrow axbX$, both X 's can be removed in 4 ways.

$$S \rightarrow axbX | abX | axb | ab$$

- Also, from $X \rightarrow aY | bY | \epsilon$, we get

$$X \rightarrow aY | bY | a | b$$

- So, after eliminating ϵ -productions:

$$S \rightarrow axbX | abX | axb | ab$$

$$X \rightarrow aY | bY | a | b$$

$$Y \rightarrow X | c$$

Step-2: Eliminate Null productions

- Unit production $Y \rightarrow X$.
- Substitute productions of X in Y .

$$X \rightarrow aY | bY | a | b$$

$$\therefore Y \rightarrow aY | bY | a | b | c$$

- After eliminating unit productions:

$$S \rightarrow axbX | abX | axb | ab$$

$$X \rightarrow aY | bY | a | b$$

$$Y \rightarrow aY | bY | a | b | c$$

Step-3: Replace terminals in mixed productions.

- Introduce new variables for terminals:

$$A \rightarrow a, B \rightarrow b$$

- Replace a by A and b by B .

$$S \rightarrow AXBX | ABX | AXB | AB$$

$$X \rightarrow AY | BY | a | b$$

$$Y \rightarrow AY | BY | a | b | c$$

Step 4: Convert long productions into binary

- The production $AXBX$ has length 4.
- Introduce new variables:

$$E \rightarrow AX, \quad F \rightarrow BX.$$

- Now,

$$AXBX = (AX)(BX) = EF$$

$$ABX = A(BX) = AF$$

$$AXB = (AX)B = EB$$

AB is already binary.

Step-5: Final CFG in CNF

$$S \rightarrow EF \mid AF \mid EB \mid AB$$

$$X \rightarrow AY \mid BY \mid a \mid b.$$

$$Y \rightarrow AY \mid BY \mid a \mid b \mid c.$$

$$E \rightarrow AX$$

$$F \rightarrow BX$$

$$A \rightarrow a$$

$$B \rightarrow b.$$

④ a) Push Down Automata (PDA)

- It is a non-deterministic finite automata equipped with a stack.
- It is used to recognize Context-Free Language.
- It is 7-tuple:

$$M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$$

where,

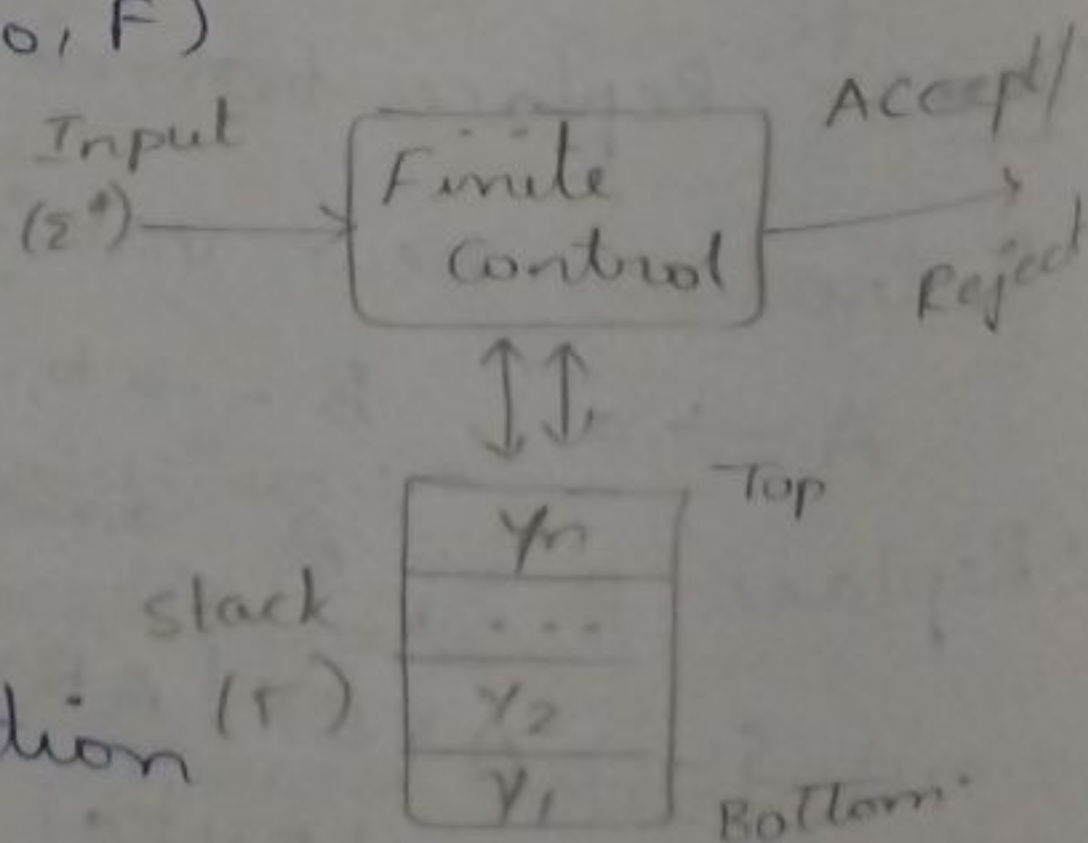
Q : Final set of states

Σ : Input alphabet

Γ : stack alphabet

δ : Transition function

$$\delta: Q \times (\Sigma \cup \{\epsilon\}) \times \Gamma \rightarrow P(Q \times \Gamma^*)$$



q_0 : Initial state ($q_0 \in Q$)

z_0 : Initial stack symbol ($z_0 \in \Gamma$)

F : Final states ($F \subseteq Q$).

- PDA reads the input string from left to right one symbol at a time.
- It looks at the current state, current input symbol & top symbol of stack.
- Based on transition δ , it can:
 - push a symbol onto the stack.
 - pop a symbol from the stack.
 - change to another state.

Eg: $L = \{a^n b^{2n} \mid n \geq 1\}$

$L = \{abb, aabbbb, aaabbbb, \dots\}$

Step-1: If input symbol 'a' then push 2 'a's

Step-2: If input symbol 'b' then stack & POP

Let string: $aabbbb \in L$

$$\delta(q_0, a, z_0) = (q_0, a_0 z_0)$$

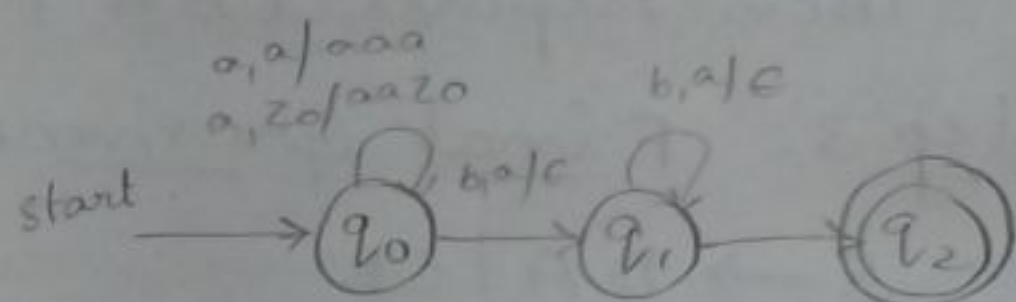
$$\delta(q_0, a, a) = (q_0, a a_0 z_0)$$

$$\delta(q_1, b, a) = (q_1, \epsilon)$$

$$\delta(q_1, \epsilon, z_0) = (q_2, z_0)$$

$$M = (Q, \Sigma, \Gamma, \delta, q_0, z_0, F)$$

$$M = (\{q_0, q_1, q_2\}, \{a, b\}, \{a, z_0\}, \delta, q_0, z_0, \{q_2\})$$



④ b) Given Grammar:

$$S \rightarrow bA \mid aB$$

$$A \rightarrow bAA \mid aS \mid a$$

$$B \rightarrow aBB \mid bS \mid a$$

Step-1: Replace terminals in mixed productions

In given grammar, terminals a and b appear with non-terminals.

- Introduce new variables for terminals.
 $C \rightarrow a, D \rightarrow b$.

- After replacement

$$S \rightarrow DA | CB$$

$$A \rightarrow DAA | CS | a$$

$$B \rightarrow CBB | DS | a$$

Step-2: Convert productions having more than 2 symbols

- In CNF, each production must have either 1 terminal (or) exactly 2 non-terminals.

- For $A \rightarrow DAA$ (length = 3)

Introduce a new variable: $E \rightarrow AA$

Then replace DAA by DE .

- For $B \rightarrow CBB$ (length = 3)

Introduce a new variable: $F \rightarrow BB$

Then replace CBB by CF .

Step-3: Final Grammar in CNF

$$S \rightarrow DA | CB$$

$$A \rightarrow DE | CS | a$$

$$B \rightarrow CF | DS | a$$

$$E \rightarrow AA$$

$$F \rightarrow BB$$

$$C \rightarrow a$$

$$D \rightarrow b$$

⑤a) Ambiguous Grammar

- A CFG is said to be ambiguous if there exists at least one string in its language that has more than one distinct parse tree (or equivalently, more than one leftmost or rightmost derivation).
- If every string has exactly one parse tree, the grammar is unambiguous.

Given Grammar:

$$S \rightarrow aAB$$

$$A \rightarrow bc \mid cd$$

$$c \rightarrow cd$$

$$B \rightarrow c \mid d$$

Step-1: Strings generated by Grammar:

Case 1: $A \rightarrow bc$

since $c \rightarrow cd \Rightarrow A \rightarrow bcd$.

$$S \Rightarrow aAB \Rightarrow a(bcd)B$$

- $B \rightarrow c \Rightarrow S \Rightarrow abcdc$
- $B \rightarrow d \Rightarrow S \Rightarrow abcd d$

Case 2: $A \rightarrow cd$

since $c \rightarrow cd \Rightarrow A \rightarrow ^ccd$.

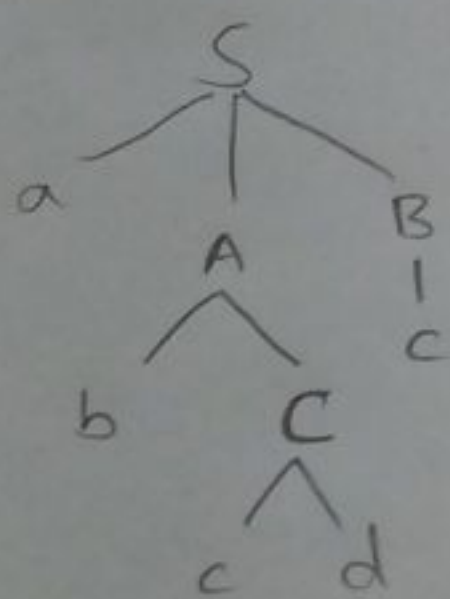
$$S \Rightarrow aAB \Rightarrow acd B$$

- $B \rightarrow c \Rightarrow S \Rightarrow acdc$
- $B \rightarrow d \Rightarrow S \Rightarrow acdd$

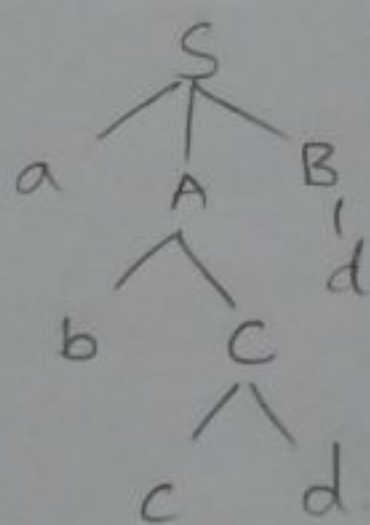
Language Generated (L) = $\{abcdc, abcd d, acdc, acdd\}$.

Step-2: parse trees for four strings

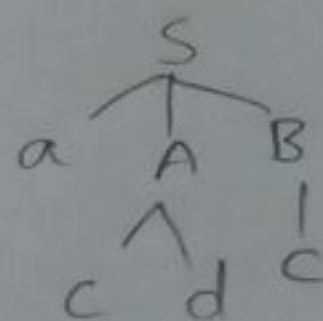
abcdc



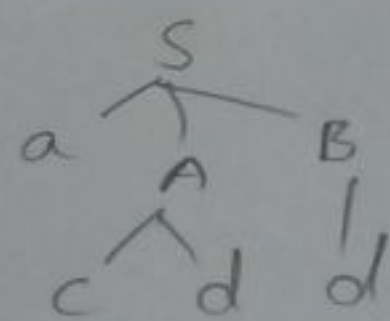
abcd d



acdc



acdd



Step-3: check for ambiguity

- For each string in $L(G)$, there is only one possible parse tree (or one leftmost derivation).
- The two alternatives for A , $A \rightarrow bc$ (gives bcd) $A \rightarrow cd$ (gives cd), produce different terminal strings. Hence, no string has 2 different parse trees. **UNAMBIGUOUS.**

⑤ b) Given Grammar $S \rightarrow AA/a, A \rightarrow SA/b$.

Step-1: Construction of PDA

Let PDA be $M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$

$Q = \{q\}$ - only one state

$\Sigma = \{a, b\}$ - input alphabet

$\Gamma = \{S, A, a, b\}$ - stack alphabet

$q_0 = q$ - initial state

$Z_0 = S$ - initial stack symbol.

$F = \emptyset$ - acceptance by empty stack.

Step-2: Transitions from production

Production

PDA Transition

$S \rightarrow AA$

$\delta(q, \epsilon, S) = (q, AA)$ - Replace S by AA

$S \rightarrow a$

$\delta(q, \epsilon, S) = (q, a)$ - Replace S by a

$A \rightarrow SA$

$\delta(q, \epsilon, A) = (q, SA)$ - Replace A by SA

$A \rightarrow b$

$\delta(q, \epsilon, A) = (q, b)$ - Replace A by b.

Step-3: Transitions for terminal matching

Input

Stack Top

PDA Transition

a

a

$\delta(q, a, a) = (q, \epsilon)$

If a is on top & next input is a, pop it.

b

b

$\delta(q, b, b) = (q, \epsilon)$

If b is on top & next input is b, pop it.

Step-4: PDA Transition Table

Current state

Input

Stack Top

Replace

Next state

q

ϵ

S

AA

q

q

ϵ

S

a

q

q

ϵ

A

SA

q

q

ϵ

A

b

q

q

a

a

ϵ

q

q

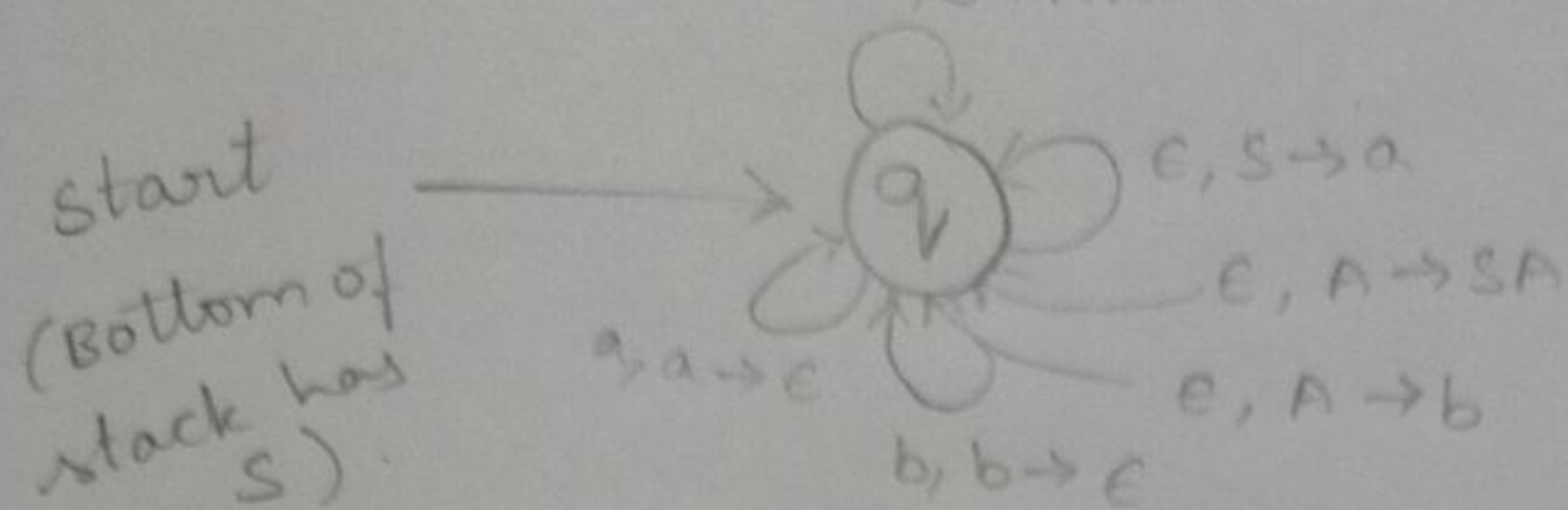
b

b

ϵ

q

Step-5: PDA state diagram



- The stack has S, using ϵ moves, PDA replaces variables on stack according to productions.
- This continues until terminal (a/b) is on top of stack.
- Then PDA matches the input symbol with stack top terminal & pops it.
- process repeats until the stack becomes empty.
- Then the string is accepted.

Step-6: Final PDA

The PDA equivalent to grammar $S \rightarrow AA|a, A \rightarrow SA|b$

$$\delta(q, \epsilon, S) = (q, AA) \mid (q, a)$$

$$\delta(q, \epsilon, A) = (q, SA) \mid (q, b)$$

$$\delta(q, a, a) = (q, \epsilon)$$

$$\delta(q, b, b) = (q, \epsilon).$$

$\therefore$ This PDA has one state, accepts by empty stack and is equivalent to given grammar.